Pseudocode outline filling in as we go

Week 1: Training the Robot to Read

- **Day 1: The Notebook.** Make a fake notebook with names, times, and "Pass" or "Fail" marks.
- **Day 2: Picking up the Book.** Teach the Robot how to open the book and look at the pages.
- **Day 3: Learning Names.** Teach the Robot to find the **IP Address** (the person's home address) and the **Time**.
- **Day 4: Counting Mistakes.** If someone tries to get in and fails, the Robot puts a tally mark next to their name.
- **Day 5: The Red Flag.** If someone fails 5 times, the Robot shouts, "Hey! This person is trying to break in!"
- **Day 6: The Secret Key.** The Robot looks for anyone trying to use the "Master Key" (that's called **Root** or **Sudo**). Only the King is allowed to have that!
- **Day 7: Practice.** You'll make sure the Robot isn't accidentally yelling at the good guys.

Week 2: Telling the King what Happened

- **Day 8: The Short Story.** The Robot learns to say: "I checked 100 people, and 3 of them were sneaky."
- **Day 9: The List of Sneaks.** The Robot makes a list of exactly who was being bad and when they did it.
- **Day 10: Saving the List.** The Robot writes this list into a special "Report Card" (a CSV file) so you can keep it forever.
- **Day 11: Fixing "Oopsies."** What if the notebook is empty? You teach the Robot not to get confused or "crash" if it finds a blank page.
- **Day 12: Writing the Rules.** You write a little instruction manual (the README) so other people know how to use your Robot.
- **Day 13: The Big Test.** You give the Robot a really messy notebook to see if it can still find the bad guys.
- **Day 14: Celebration!** You show off your Robot Guard and put it on your GitHub for everyone to see.

Let’s get the Robot Guard to pick up the notebook and read the first line!

A special Python trick called with open(). This tells the computer: *"Hey, open this file, let me look at it, and then close it when I'm done so I don't leave a mess."*

Create your "Notebook" (The Log File)

Before the robot can read, it needs something to look at. auth.log in the same folder where you'll save your Python code.

`"r"`: This stands for **Read**. It tells the robot, "Just look at the words, don't write anything new in the book yet."
`.strip()`: Logs usually have an invisible "Enter" key at the end of every line. This command trims those off so the output looks nice and clean in your terminal.
`try/except`: This is like a safety net. If you forget to make the file, the robot won't throw a "tantrum" (error); it will just calmly tell you the file is missing.

capstone > auth.log

```
Feb 13 19:00:01 gatekeeper sshd[123]: Failed password for root from 192.168.1.50
Feb 13 19:05:10 gatekeeper sshd[124]: Accepted password for steelworker51 from 192.168.1.10
Feb 13 19:10:22 gatekeeper sshd[125]: Failed password for root from 192.168.1.50
```

The Clock (`Feb 13 19:00:01`): This is the **When**. It tells us exactly what time the person tried to get in.
The Guard's Name (`sshd`): This is the **Who is watching**. `sshd` is a "Digital Guard" that handles people trying to connect to the computer from far away.
The Red Flag (`Failed password`): This is the **Action**. This tells us someone guessed the password wrong. One mistake is okay, but 50 mistakes means someone is trying to break in!
The Secret Name (`root`): This is the **Target**. `root` is the name of the "King" of the computer. Bad guys always try to log in as `root` because they want total control.
The Home Address (`192.168.1.50`): This is the **Where**. Every computer on the internet has a number address (an IP Address). This tells us exactly where the "Bad Guy" is sitting.

```
capstone > log_analyzer.py > ...
# The Robot Guard's First Words

# 1. We tell the robot which file to look at
log_file_name = "auth.log"

~/Documents/code you/capstone/log_analyzer.py ng up! ---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            # The robot "shouts" what it sees
            print(f"I see a line: {line.strip()}")

except FileNotFoundError:
    print("Oops! I can't find the 'auth.log' file. Did you make it yet?"

print("--- Robot Guard is finished reading for now! ---")
```

```
(.venv) heatherworstell@Heathers-MacBook-Air code you % ls
Design a Secure System Architecture     week 6
capstone                                week10
week 3 python                           wireshark
(.venv) heatherworstell@Heathers-MacBook-Air code you % cd capstone
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is starting up! ---
I see a line: Feb 13 19:00:01 gatekeeper sshd[123]: Failed password for root from 192.168.1.50
I see a line: Feb 13 19:05:10 gatekeeper sshd[124]: Accepted password for steelworker51 from 192.168.1.10
I see a line: Feb 13 19:10:22 gatekeeper sshd[125]: Failed password for root from 192.168.1.50
--- Robot Guard is finished reading for now! ---
(.venv) heatherworstell@Heathers-MacBook-Air capstone %
```

It picked up the book (`open`)

It looked at every line (`for line in notebook`)

It shouted what it saw (`print`)

It cleaned up its mess (The `with` part closed the book automatically)

```python
# The Robot Guard's First Words

# 1. We tell the robot which file to look at
log_file_name = "auth.log"

print("--- Robot Guard is now HUNTING! ---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            # We tell the robot: ONLY shout if you see the word 'Failed'
            if "Failed" in line:
                print(f"🚨 ALERT! Found a Bad Guy: {line.strip()}")
            else:
                # The robot stays quiet for the good guys
                pass

except Exception as e:
    print(f"The robot had a boo-boo: {e}")

print("--- Hunting finished! ---")
```

The Sifter: Think of the `auth.log` like a bucket of sand mixed with gold nuggets.
The Sand: These are the "Accepted password" lines (the Good Guys). They are boring, so we let them fall through the holes in the sifter.
The Gold: These are the "Failed password" lines. These are "heavy" and important, so they stay in the sifter.
The Command: When we write `if "Failed" in line:`, we are telling the robot: *Check the line. Does it have the 'Failed' stamp on it? If yes, keep it and shout! If no, throw it away.*

```
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is starting up! ---
I see a line: Feb 13 19:00:01 gatekeeper sshd[123]: Failed password for root from 192.168.1.50
I see a line: Feb 13 19:05:10 gatekeeper sshd[124]: Accepted password for steelworker51 from 192.168.1.10
I see a line: Feb 13 19:10:22 gatekeeper sshd[125]: Failed password for root from 192.168.1.50
--- Robot Guard is finished reading for now! ---
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is now HUNTING! ---
🚨 ALERT! Found a Bad Guy: Feb 13 19:00:01 gatekeeper sshd[123]: Failed password for root from 192.168.1.50
🚨 ALERT! Found a Bad Guy: Feb 13 19:10:22 gatekeeper sshd[125]: Failed password for root from 192.168.1.50
--- Hunting finished! ---
```

add a "Counter" variable. Think of this like a scoreboard.

```python
# The Robot Guard's First Words

# 1. We tell the robot which file to look at
log_file_name = "auth.log"

# 1. Start the scoreboard at zero
failed_count = 0

print("--- Robot Guard is now HUNTING! ---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            if "Failed" in line:
                # 2. Every time we see 'Failed', add 1 to the score
                failed_count = failed_count + 1
                print(f"🚨 ALERT! Bad guy spotted!")

except Exception as e:
    print(f"The robot had a boo-boo: {e}")

# 3. At the very end, the robot tells us the final score
print("--- Hunt finished! ---")
print(f"Total Bad Guys found: {failed_count}")
```

```
--- Hunting finished! ---
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is now HUNTING! ---
🚨 ALERT! Bad guy spotted!
🚨 ALERT! Bad guy spotted!
--- Hunt finished! ---
Total Bad Guys found: 2
(.venv) heatherworstell@Heathers-MacBook-Air capstone %
```

upgrade the robot's brain from a simple "tally counter" to a **Dictionary**.

```python
# 1. Create the empty mailboxes (Dictionary)
ip_counts = {}
# 2. We tell the robot which file to look at
log_file_name = "auth.log"

print("--- Robot Guard is sorting the mail! ---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            if "Failed" in line:
                # finds the IP at the end of the line
                # For now, we are just using the one from your auth.log
                ip_address = line.split()[-1]

                # 2. Check if we already have a mailbox for this IP
                if ip_address in ip_counts:
                    ip_counts[ip_address] += 1 # Add a marble
                else:
                    ip_counts[ip_address] = 1  # Create a new mailbox with 1 marble

except Exception as e:
    print(f"The robot had a boo-boo: {e}")

# 3. Show the final mailbox counts
print("--- Final Report ---")
for ip, count in ip_counts.items():
    print(f"IP Address {ip} failed {count} times.")
```

```
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is sorting the mail! ---
--- Final Report ---
IP Address 192.168.1.50 failed 2 times.
```

teach the robot to look for people trying to use "Boss Mode" (sudo)

```python
# 1. Create the empty mailboxes (Dictionary)
ip_counts = {}
# 2. We tell the robot which file to look at
log_file_name = "auth.log"

print("--- Robot Guard is sorting the mail! ---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            if "Failed" in line:
                # finds the IP at the end of the line
                # For now, we are just using the one from your auth.log
                ip_address = line.split()[-1]

                # 2. Check if we already have a mailbox for this IP
                if ip_address in ip_counts:
                    ip_counts[ip_address] += 1 # Add a marble
                else:
                    ip_counts[ip_address] = 1  # Create a new mailbox with 1 marble

except Exception as e:
    print(f"The robot had a boo-boo: {e}")

# 3. Show the final mailbox counts
print("--- Final Report ---")
for ip, count in ip_counts.items():
    print(f"IP Address {ip} failed {count} times.")
```

```
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python log_analyzer.py
--- Robot Guard is checking for stolen keys!---
--- Final Report ---
IP Address 192.168.1.50 failed 2 times.
--- Boss Mode Attempts ---

🚨 WARNING: Found 2 attempts to use Boss Mode!
Attempt: Feb 13 19:00:01 gatekeeper sshd[123]: Failed password for root from 192.168.1.50
Attempt: Feb 13 19:10:22 gatekeeper sshd[125]: Failed password for root from 192.168.1.50
(.venv) heatherworstell@Heathers-MacBook-Air capstone %
```

a robot that:

1. **Reads** the book.
2. **Filters** the bad guys.
3. **Counts** the bullies.
4. **Protects** the Master Keys.

```python
# 1. Create the empty mailboxes (Dictionary)

# 2. We tell the robot which file to look at
log_file_name = "auth.log"
# 1. New list to hold the names of people using 'Boss Mode'
boss_mode_attempts = []
total_lines = 0
print("--- Robot Guard is generating the Final Report!---")

# 2. We open the notebook carefully
try:
    with open(log_file_name, "r") as notebook:
        # 3. We tell the robot to read it line by line
        for line in notebook:
            total_lines += 1

            if "Failed" in line:
                # finds the IP at the end of the line
                # For now, we are just using the one from your auth.log
                ip_address = line.split()[-1]
                ip_counts[ip_address] = ip_counts.get(ip_address, 0) + 1
                # 2. NEW: Check if someone is trying to use 'sudo' or 'root'
            if "sudo" in line or "root" in line:
                boss_mode_attempts.append(line.strip())

    #--- THE SUMMARY REPORT SECTION ---
    print("\n" + "-" * 30)
    print("SECURITY SUMMARY REPORT")
    print("-" * 30)
    print(f"Total lines scanned: {total_lines}")
    print(f"Unique IPs Flagged: {len(ip_counts)}")
    print(f"Boss Mode Threats Detected: {len(boss_mode_attempts)}")
    print("-" * 30)
```

```python
    #only show the details if we actually found something ---
    if ip_counts:
        print("TOP OFFENDERS:")
        for ip, count in ip_counts.items():
            print(f"- {ip}: {count} failed attempts")

    print("=" * 30)

except Exception as e:
    print(f"The robot had a boo-boo: {e}")
```

```
------------------------------
SECURITY SUMMARY REPORT
------------------------------
Total lines scanned: 3
Unique IPs Flagged: 1
Boss Mode Threats Detected: 2
------------------------------
TOP OFFENDERS:
- 192.168.1.50: 2 failed attempts
==============================
(.venv) heatherworstell@Heathers-MacBook-Air capstone %
```

`"=" * 30`: This is just a fun Python trick to draw a straight line of 30 equal signs. It makes the report look like it was printed from a professional machine.
`"\n"`: This tells the robot to "Hit the Enter key" to create a blank line so the text isn't all squished together.
A show how to take scary technical data and turn it into a clear "All Clear" or "We have a problem" message, helps showcase the communication skills us as aspiring cybersecurity experts should have.

We need to `import csv` at the very top. This gives the robot a "Pen and Paper" to write with.

```python
import csv  # 1. Give the robot a pen
# The Robot Guard's First Words
# 1. Create the empty mailboxes (Dictionary)
ip_counts = {}
# 2. We tell the robot which file to look at
log_file_name = "auth.log"
# 1. New list to hold the names of people using 'Boss Mode'
boss_mode_attempts = []
total_lines = 0
print("--- Robot Guard is generating the Final Report!---")

# 2. Open a NEW file to write our spreadsheet
try:
    with open('security_report.csv', 'w', newline='') as spreadsheet:
        writer = csv.writer(spreadsheet)

        # 3. Write the "Header" (The top of the columns)
        writer.writerow(["IP Address", "Number of Fails"])

        # 4. Write the data
        for ip, count in ip_counts.items():
            writer.writerow([ip, count])

    print("--- Spreadsheet 'security_report.csv' has been created! ---")

except Exception as e:
    print(f"The robot had a boo-boo: {e}")
```

In Python, `import` is how we get expansion packs. The `csv` pack teaches the robot how to draw straight lines and columns so the data stays organized.

By writing a **CSV file**, the robot is carving its findings into **stone**.

- You can email this file to your boss.
- You can save it to prove you found a hacker last Tuesday.
- You can use it to make cool charts and graphs in Excel.

"breaking" your script on purpose to see the Safety Net work? (Just rename `auth.log` to `secret.log` and run the script!)

```
==============================
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python3 log_analyzer.py
--- Robot Guard is generating the Final Report!---
--- Spreadsheet 'security_report.csv' has been created! ---
(.venv) heatherworstell@Heathers-MacBook-Air capstone % ls
auth.log                google docs             log_analyzer.py         security_report.csv
```

1. **The Bully:** It should see that `10.0.0.5` failed **4 times**.
2. **The Stranger:** It should see that `172.16.0.42` failed **1 time**.
3. **The Master Key:** It should flag that line where `dale_m` used **sudo**.

```
capstone > auth.log
Mar 05 08:00:01 gatekeeper sshd: Accepted password for boss_man from 192.168.1.2
Mar 05 08:01:15 gatekeeper sshd: Failed password for root from 10.0.0.5
Mar 05 08:01:16 gatekeeper sshd: Failed password for root from 10.0.0.5
Mar 05 08:01:17 gatekeeper sshd: Failed password for root from 10.0.0.5
Mar 05 08:05:00 gatekeeper sshd: Accepted password for dale_m from 192.168.1.15
Mar 05 08:10:22 gatekeeper sudo: user dale_m : TTY=pts/0 ; PWD=/home/dale_m ; USER=root ; COMMAND=/usr/bin/ls
Mar 05 08:12:00 gatekeeper sshd: Failed password for admin from 172.16.0.42
Mar 05 08:15:45 gatekeeper sshd: Failed password for root from 10.0.0.5
```

```
capstone > security_report.csv > data
IP Address,Number of Fails
10.0.0.5,4
172.16.0.42,1
```

```
# --- STEP 2: (THE TIME-OUT RULE) ---
print("\n🛡️ SECURITY ACTION TAKEN:")
for ip, count in ip_counts.items():
    if count >= 3:
        print(f"🚫 BLOCKING IP: {ip} (Too many fails: {count})")
```

```
SyntaxError: invalid syntax
(.venv) heatherworstell@Heathers-MacBook-Air capstone % python3 log_analyzer.py
--- Robot Guard is starting the hunt! ---

🛡️ SECURITY ACTION TAKEN:
🚫 BLOCKING IP: 10.0.0.5 (Too many fails: 4)

✅ Everything worked perfectly! CSV created.
--- Hunt Finished ---
```

"Hi, I've completed Option A: The Log Analyzer. My script successfully parses system logs, identifies multiple failed login attempts by IP address, and flags privilege escalation (sudo/root) indicators. I have also implemented error handling for missing files and included a CSV export feature for professional reporting. Everything is documented in the attached README."